

Week 9: Unbonded Knapsack Problem

411025020

Questions

1. We can solve above problem by dynamic programming approach, what's the transition function? Please write down your answer in recursively way.

```
dp[i][j] = max(dp[i-1][j] , dp[i][j-w[i]]+v[i])
```

2. Traditional DP algorithm will construct an $n \times W$ table to keep subproblem solutions. We can improve the space complexity by reduce the size of cache table. Please design an algorithm to solving unbounded knapsack problem with space complexity $O(W)$

wv是權重與價值、mx是最大負荷

```
vector<pair<int,int>> wv;
vector<int> dp(mx+1,-1);
dp[0]=0;
for(int i=0;i<wv.size();i++)
{
    for(int j=wv[i].first;j<=mx;j++)
    {
        if(dp[j-wv[i].first]!=-1 && dp[j-wv[i].first]+wv[i].second>dp[j])
        {
            dp[j]=dp[j-wv[i].first]+wv[i].second
        }
    }
}
```

若想找到答案只需從尾端找到第一個不是-1的值

```
for(int i=wv.size();i>=0;i--)
{
    if(dp[i]!=-1)
    {
        return dp[i];
    }
}
```

3. Design an algorithm to reconstruct the combination of items with $O(n \times W)$ table, and analyzing its time complexity.

```

int dp[wv.size()+1][mx+1];
for(int i=0;i<wv.size()+1;i++)
    dp[i][0]=0;
for(int j=0;j<mx+1;j++)
    dp[0][j]=0;
for(int i=1;i<wv.size()+1;i++)
    for(int j=w[i];j<mx+1;j++)
        dp[i][j] = max(dp[i-1][j] , dp[i][j-w[i]]+v[i])

```

時間複雜度為 $O(n \times 2^{\log_2 W})$

4. Same to Q3, but doing things with $O(W)$.

時間複雜度亦為 $O(n \times 2^{\log_2 W})$

5. Please solve <https://oj.csie.ndhu.edu.tw/problem/ALG13>

C++

```

#include<iostream>
#include<vector>
using namespace std;
int main()
{
    int burger[2], t;

    while (cin>>burger[0]>>burger[1]>>t)
    {
        int i, j;
        vector<int> eat(t+1,-1);
        eat[0] = 0;

        for (i = 0; i < 2; i++)
        {
            for (j = burger[i]; j <= t; j++)
            {
                if (eat[j-burger[i]] != -1 && eat[j-burger[i]] + 1 > eat[j])
                {
                    eat[j] = eat[j - burger[i]] + 1;
                }
            }
        }
        for (i = t; i >= 0; i--)
        {
            if (eat[i] != -1)
            {
                cout<<eat[i]<<' ';
                break;
            }
        }
    }
}

```

```
        if (i != t)
            cout<<t - i;
        cout<<endl;
    }
    return 0;
}
```